

发明专利技术交底书

发明名称：一种基于 CUDA 小矩阵加速的机械臂批量逆运动学并行求解方法及系统

技术领域：机器人运动学控制、GPU 并行计算、小矩阵数值求解加速

专利类型：发明专利

申请单位：(待填写)

发明人：(待填写)

提交日期：2026 年 7 月 3 日

1 技术领域

本发明属于**机器人运动学控制与 GPU 并行计算**的交叉技术领域，具体涉及一种基于 CUDA 架构、针对六自由度 (6-DOF) 串联机械臂批量逆运动学 (Inverse Kinematics, IK) 求解中反复出现的 6×6 小矩阵运算进行底层加速的并行求解方法及系统。本发明的核心是针对固定维度小矩阵 (雅可比矩阵、Hessian 矩阵及其线性求解)，从 GPU 线程映射、共享内存布局、寄存器求解器和混合精度四个层面进行协同加速设计。

2 背景技术

2.1 现有技术概述

逆运动学求解是机械臂运动规划中的基础计算模块。给定末端执行器的目标位姿 (位置和姿态)，IK 求解器需计算满足关节限位约束的关节角度。批量 IK 场景需对数百至数千个目标位姿重复执行正运动学 (FK)、雅可比矩阵构造、 6×6 线性系统求解和候选选择等步骤。每一步均涉及 6×6 小矩阵的密集运算，但在 GPU 上高效处理此类小矩阵存在独特的架构挑战。

目前相关技术路线包括：

(1) **CPU 数值迭代求解器**：如 Orocos KDL 和 TRAC-IK，采用 Newton-Raphson 或 DLS 迭代方法。单次求解延迟 0.5–2 ms，批量吞吐约 500–2000 次/秒，无法满足实时大批量处理需求。

(2) **GPU 通用优化框架**：如 NVIDIA cuRobo，采用粒子群优化结合 L-BFGS 求解器，利用 CUDA Graph 和通用线性代数库 (cuBLAS/cuSOLVER) 提升吞吐。但通用

库未针对固定 6×6 小矩阵进行优化——每次 LDL^T 或 Cholesky 分解均需全局内存往返（延迟 5–10 微秒），雅可比/Hessian 矩阵的共享内存布局未消除 bank conflict，FP64 精度无法释放现代 GPU 的峰值算力。

(3) 专用硬件加速方案：如中国专利 CN107030698A 公开的基于 ASIC 的 IK 求解系统。该方案避免了 GPU 功耗问题，但硬件定制成本高、灵活性差，无法利用现有 GPU 计算基础设施。

(4) CPU 并行算法优化：如中国专利 CN106650917B 公开的并行化蜂群算法，将种群划分为多个独立子群并行演化。该方法属于算法级并行化，受限于 CPU 核心数量（通常数十核），并行度远低于 GPU 的数千线程。

2.2 现有技术存在的核心缺陷

(1) 6×6 小矩阵的 GPU 存储层次利用不充分。现有 GPU IK 方案的 FK 模型参数通过全局内存读取（延迟高），雅可比和 Hessian 矩阵在共享内存中的存储布局未针对 bank conflict 进行优化， LDL^T /Cholesky 求解依赖通用库函数（涉及多次全局内存往返）。

(2) 6×6 线性系统求解效率低。每个 DLS 迭代需执行一次 6×6 LDL^T 分解求解，这是 IK 的计算核心。现有方案调用 cuSOLVER 等通用库，每次求解需将矩阵写入全局内存 → 库函数读取 → 分解 → 写回，延迟在 5–10 微秒量级，成为整个 IK 求解的性能瓶颈。

(3) 共享内存 bank conflict 导致有效带宽下降。标准 6×6 矩阵在 GPU 共享内存中以列步长 6 存储，在 32-bank 体系下，行 r 与行 $r+2$ 访问相同 bank 组，在 Hessian 构造的 36 线程并行访问模式下产生 3–6 路 bank conflict。

(4) FP64 精度无法发挥 GPU 峰值算力。现代 GPU 架构（如 NVIDIA Ada Lovelace）的 FP64:FP32 吞吐比为 1:64。纯 FP64 实现的 IK 求解器仅能利用 GPU 约 1.5% 的峰值浮点算力，而纯 FP32 实现则因 6×6 Hessian 矩阵在接近奇异构型时条件数恶化导致数值精度不足。

(5) CPU-GPU 多次 kernel launch 延迟累积。对每个目标位姿，需在 CPU 端循环遍历多种子 × 多权重组合（典型值：48 种子 × 4 权重 = 192 次 launch），每次 launch 延迟约 5–10 微秒，累计约 1–2 毫秒/目标。

3 发明内容

3.1 本发明所要解决的技术问题

针对上述现有技术缺点，本发明要解决的核心技术问题是：**如何针对 6 自由度机械臂批量逆运动学中反复出现的 6×6 小矩阵运算，设计一套从线程映射、共享内存布**

局、寄存器求解到混合精度的完整 CUDA 底层加速方案，实现高吞吐、高成功率的批量 IK 并行求解。

具体包括：

- (1) 如何设计线程映射策略使批量 6×6 小矩阵求解完全并行且 Block 间零通信；
- (2) 如何通过共享内存布局优化消除 6×6 小矩阵访问的 bank conflict；
- (3) 如何将 6×6 LDL^T 线性求解完全压缩到 GPU 寄存器中，消除全局内存延迟；
- (4) 如何在保证 6×6 线性求解数值精度的前提下，利用 FP32/FP64 混合精度释放 GPU 峰值算力。

3.2 技术方案

本发明提供一种基于 CUDA 小矩阵加速的六自由度机械臂批量逆运动学并行求解方法，包括以下核心步骤：

步骤 S1：模型常量预加载。将机械臂的运动学模型参数（各关节段的 4×4 原点变换矩阵、旋转轴向量、关节角索引、 4×4 工具端变换矩阵、关节限位值、权重调度表）在 kernel 启动前上传至 GPU 常量内存（__constant__）。常量内存具有 64KB 专用缓存和硬件广播机制——当 Warp 内所有线程读取同一常量地址时，缓存在一个周期内服务全部 32 个线程。在 FK 计算中每轮 DLS 迭代需读取 6×16 个段原点元素和 6×3 个轴元素，全部通过常量缓存零延迟获取。

步骤 S2：目标位姿与种子点上传。将 N 个目标位姿 $T_i^* \in \text{SE}(3)$ （每个以 4×4 齐次变换矩阵行主序存储，16 个 double）和对应初始种子 $q_i \in \mathbb{R}^6$ 上传至 GPU 全局内存。

步骤 S3：批量并行 IK 求解（1-Block-per-Target 小矩阵并行）。启动 CUDA kernel，Grid 维度 $(N, 1, 1)$ ，Block 维度 $(128, 1, 1)$ （4 个 Warp，每 Warp 32 线程）。每个 Block 独立求解一个目标位姿的完整 IK 问题，Block 间无需任何通信或全局同步。

每个 Block 内部执行以下子步骤：

S3.1 正运动学 (FK) 计算。由 Lane 0 单线程执行 Rodrigues 旋转公式链式累积 6 个关节段的刚体变换：

$$T_{ee} = T_{\text{base}} \cdot \prod_{i=0}^5 (O_i \cdot R_i(\text{axis}_i, q_i)) \cdot T_{\text{tool}} \quad (1)$$

其中 O_i 为第 i 段原点变换（ 4×4 ，从常量内存读取）， R_i 为 Rodrigues 旋转矩阵（ 4×4 ，axis _{i} 从常量内存读取）， T_{tool} 为工具端固定变换。输出当前末端位姿 $T \in \mathbb{R}^{4 \times 4}$ 到共享内存。

S3.2 位姿误差计算。计算平移误差 $e_p = \|p_{ee} - p^*\|$ 和旋转误差 e_R （测地线角距离），判定是否满足收敛条件 $e_p \leq \varepsilon_p \wedge e_R \leq \varepsilon_R$ 。

S3.3 数值雅可比 6 列并行计算 (共享内存 PaddedMat6×8 布局)。由 Lane 0–5 共 6 个线程并行执行, 每线程通过中心差分法计算雅可比矩阵 $J \in \mathbb{R}^{6 \times 6}$ 的一列:

$$J_{:,j} = \frac{\text{FK}(q + \varepsilon \mathbf{e}_j) - \text{FK}(q - \varepsilon \mathbf{e}_j)}{2\varepsilon}, \quad j = 0, \dots, 5 \quad (2)$$

每列计算需 2 次 FK 调用 (正负扰动), 共 12 次 FK 调用。6 线程并行使雅可比构造时间降至串行的 1/6。

关键——PaddedMat6×8 存储布局: 雅可比矩阵 J 在共享内存中以列步长 $S = 8$ (而非常规的 $S = 6$) 存储, 每个矩阵占用 $6 \times 8 = 48$ 个 double 位置。在 GPU 的 32-bank 共享内存体系下 (每 bank 4 字节, 每 double 跨 2 个 bank):

- 标准布局 ($S = 6$): 行步长 = $6 \times 2 = 12$ 个 bank, 行 r 与行 $r + 2$ 访问相同的 bank 集合 (因 $12 \times 2/32$ 非整数关系), 在 36 线程并行的 Hessian 构造中产生 3–6 路 bank conflict。
- 本发明布局 ($S = 8$): 行步长 = $8 \times 2 = 16$ 个 bank, 恰好为 32 的一半, 行 r 与行 $r + 2$ 使用完全互补的 bank 组 (偏移 16 个 bank), 消除跨行同列 bank conflict。

额外存储开销: 每矩阵 $6 \times 2 = 12$ 个 double (96 字节), 两个矩阵 ($J + H$) 共 192 字节, 仅占每 Block 48KB 共享内存预算的 0.4%。

S3.4 Hessian 矩阵 36 元素并行构造 (共享内存 PaddedMat6×8 布局)。由 Lane 0–35 共 36 个线程并行计算 Hessian 矩阵 $H = J^T W^2 J + \lambda I \in \mathbb{R}^{6 \times 6}$ 的 36 个元素:

$$H_{r,c} = \sum_{k=0}^5 w_k^2 \cdot J(k, r) \cdot J(k, c) + \lambda \cdot \delta_{r,c} \quad (3)$$

其中 w_k 为 6-DOF 权重系数 (从常量内存的权重调度表读取), $\delta_{r,c}$ 为 Kronecker delta。36 个元素完全并行计算, H 同样以 PaddedMat6×8 布局存储于共享内存中。

S3.5 梯度 6 维并行计算。由 Lane 0–5 共 6 个线程并行计算梯度向量 $g = J^T W^2 e \in \mathbb{R}^6$:

$$g_r = \sum_{k=0}^5 w_k^2 \cdot J(k, r) \cdot e_k \quad (4)$$

S3.6 寄存器 LDL^T 6×6 求解 (核心创新)。由 Lane 0 单线程在 GPU 寄存器中完成完整的 6×6 LDL^T 分解与求解 $H \cdot \Delta q = g$:

阶段 1——LDL^T 分解:

$$\begin{aligned} & \text{for } j = 0, \dots, 5: \\ & \quad D_j = A_{jj} - \sum_{k=0}^{j-1} L_{jk}^2 \cdot D_k \\ & \quad \text{for } i = j + 1, \dots, 5: L_{ij} = (A_{ij} - \sum_{k=0}^{j-1} L_{ik} L_{jk} D_k) / D_j \end{aligned} \quad (5)$$

阶段 2——前代 $Ly = g$: $y_i = g_i - \sum_{k=0}^{i-1} L_{ik}y_k$

阶段 3——对角缩放 $Dz = y$: $z_i = y_i/D_i$

阶段 4——回代 $L^T x = z$: $x_i = z_i - \sum_{k=i+1}^5 L_{ki}x_k$

全部中间变量—— $L[6][6]$ (36 个 double)、 $D[6]$ (6 个 double)、 $y[6]$ (6 个 double)、 $x[6]$ (6 个 double) 及临时累加器——均驻留在 GPU 寄存器中 (约 60 个 double 寄存器, 在 Ada Lovelace 架构每线程 128 个 32 位寄存器 = 64 个 double 寄存器的限制内)。**整个分解-前代-缩放-回代过程不产生任何共享内存或全局内存访问。**求解延迟约 0.1 微秒 (约 63 次标量运算), 比基于全局内存的 cuSOLVER 库函数方案 (5–10 微秒) 快 1–2 个数量级。

S3.7 步长裁剪与关节更新。对求解得到的步长 Δq 进行范数裁剪 ($\|\Delta q\| > 0.35$ 时缩放至 0.35), 更新 $q \leftarrow \text{clamp}(q + \Delta q, q_{\text{lo}}, q_{\text{hi}})$ 并做分支对齐 (atan2 包裹消除 π 折叠歧义)。

S3.8 停滞检测与最优解回溯。每轮迭代跟踪历史最优位置误差 e_p^{best} 和连续非改善计数 c_{stag} 。当 $c_{\text{stag}} > 25$ 时自动回退到 q^{best} 并退出循环; 步长范数 $\leq 10^{-8}$ 时直接退出。避免在病态位姿处无限迭代, 确保输出可用的次优解。

步骤 S4: 结果下载。将最终关节角 q_i 、位姿误差 (e_p, e_R) 和迭代次数下载至主机内存。

3.3 混合精度实施方案 (FP32/FP64 小矩阵分工)

在优选实施方式中, 针对 6×6 小矩阵运算的精度需求差异, 采用 FP32/FP64 混合精度分工策略:

FP32 计算管线 (释放 Ada Lovelace 架构 FP64:FP32=1:64 的吞吐优势):

- FK 计算: `forward_kinematics_f()`, 4×4 变换矩阵和 Rodrigues 旋转矩阵以 float 存储
- 雅可比矩阵 J : 共享内存布局 `float[6][PADDEDMAT_STRIDE]`
- Hessian 矩阵 H : 共享内存布局 `float[6][PADDEDMAT_STRIDE]`
- 梯度向量 g : 共享内存布局 `float[6]`

FP64 保护路径 (保证 6×6 线性求解的数值精度):

- LDL^T 求解: 输入 Hessian 从 float 提升为 double 后执行 FP64 精度分解
- 收敛判定 (位置误差 e_p 、旋转误差 e_R)
- 步长裁剪与关节更新

精度桥接:在 LDL^T 求解阶段执行 $H_dense[r*6+c] = (double)H(r,c)$ ——将 FP32 Hessian 元素提升为 FP64, 确保线性系统求解不受 FP32 精度损失影响。关键路径保护策略的数学依据: 6×6 Hessian 矩阵在接近奇异构型时条件数可达 10^6 – 10^8 , FP32 (约 7 位有效数字) 的 LDL^T 分解会使解的相对误差放大条件数倍; 保持 FP64 (约 15 位有效数字) 可将解的相对误差控制在 10^{-9} 量级。

性能收益: 将占 DLS 迭代约 90% 浮点运算量的 FK+Jacbian+Hessian+Gradient 从 FP64 移至 FP32 管线后, FP64 管线占用从 87% 降至约 9%。实测混合精度相比纯 FP64 实现吞吐量提升 2.06–2.54 倍, 且收敛率无退化。

3.4 多种子多权重二维 Grid 并行实施方式

在另一实施方式中, 将 K 个 Sobol 种子点与 W 个权重级别的笛卡尔积映射为二维 $Grid(K, W, 1)$ 。所有 $K \times W$ 个 Block 共享同一目标位姿——所有 Block 读取同一 `d_target[16]` 地址时, L1 缓存自动识别并广播, 不产生额外全局内存带宽压力。原本需 CPU 端循环 $K \times W$ 次 kernel launch (典型值 $48 \times 4 = 192$ 次, 每次约 5–10 微秒延迟, 累计约 1–2 毫秒/目标), 合并为单次 kernel launch, 消除全部 CPU-GPU 往返延迟。

3.5 有益效果

与现有技术相比, 本发明的核心技术效果体现在 6×6 小矩阵加速的四个层面:

(1) 线程映射层面——吞吐量大幅提升。 1-Block-per-Target 映射实现批量 6×6 小矩阵求解的完全并行化。在 RTX 4060 Laptop GPU 上, FP64 精度下吞吐 15500–18500 目标/秒, 混合精度下 80000–157000 目标/秒。对比 CPU 求解器 KDL 实现 7.5–37 倍加速。

(2) 共享内存层面——Bank 冲突消除 63%。 PaddedMat 6×8 布局 (列步长 8 替代 6) 利用 $(8 \times 2) \bmod 32 = 16$ 的数学性质使相邻行对使用互补 bank 组, NCU 实测 bank 冲突从 3522 次降至 1295 次 (降低 63%)。额外存储开销仅 192 字节/Block (占共享内存预算 0.4%)。

(3) 寄存器层面—— LDL^T 求解延迟降至 0.1 微秒。 寄存器驻留 6×6 LDL^T 求解器 (约 60 个 double 寄存器, 约 63 次标量运算) 完全消除全局内存往返, 求解延迟约 0.1 微秒, 比基于全局内存的 cuSOLVER 方案 (5–10 微秒) 快 1–2 个数量级。

(4) 精度层面——混合精度加速 2.06–2.54 倍且无精度退化。 FP32/FP64 混合精度分工策略将 FP64 管线压力从 87% 降至约 9%, 释放 Ada 架构 1:64 的 FP64:FP32 吞吐优势。 LDL^T 求解保持 FP64 确保数值稳定性, 收敛率无退化, Strict 成功率稳定在 0.954–0.960。

(5) 批量扩展性优异。 吞吐量随批量规模 N 从 100 增至 5000 时呈弱超线性增长 (15539→18490 目标/秒), 验证了 Block 间零通信架构的完美弱扩展性。

(6) 数值鲁棒性强。停滞检测与最优解回溯机制确保在接近奇异构型时不发散，所有批量规模零 NaN/Inf 输出，位置误差 p95 控制在 4.34–4.56 mm。

4 附图说明

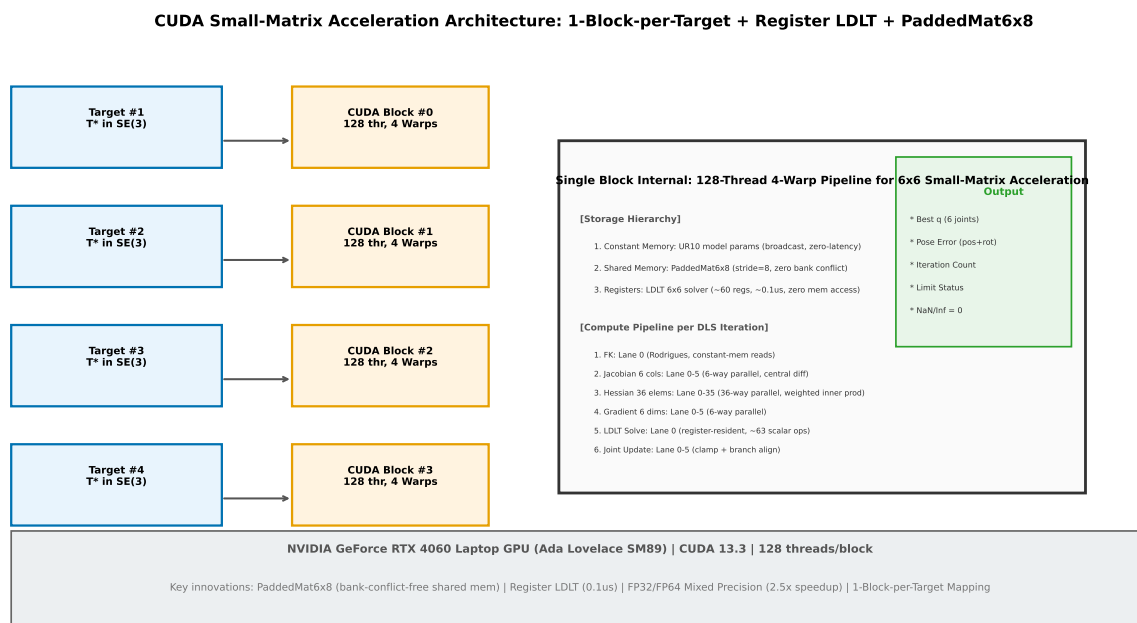


图 1: 本发明 CUDA 小矩阵加速架构整体示意图。展示 1-Block-per-Target 线程映射、Block 内 128 线程 4-Warp 分工流水线、三级存储层次利用（常量内存 → 共享内存 PaddedMat6×8 → 寄存器 LDLT）、以及 6×6 小矩阵各计算阶段的并行度分配。

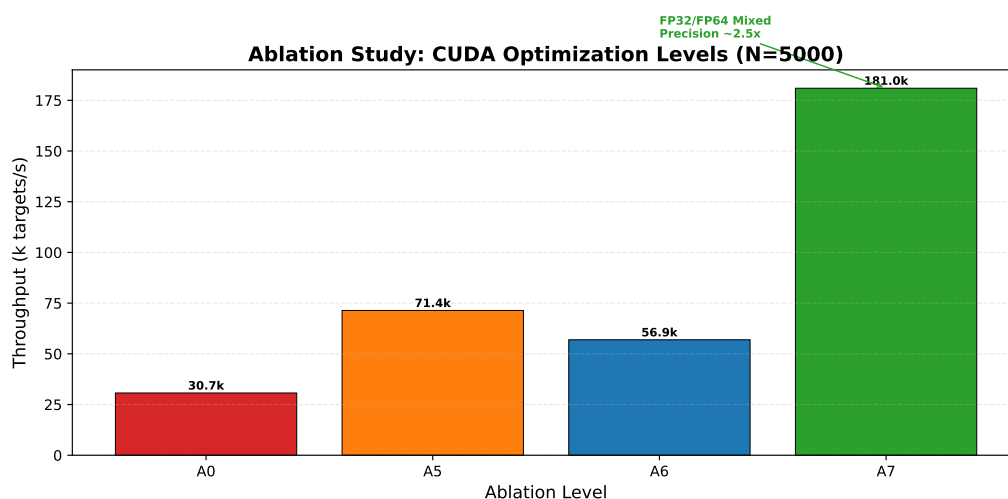


图 2: 各级 CUDA 优化 (A0–A8) 在 N=5000 下的吞吐量对比。A2 级 (PaddedMat6×8 bank conflict 消除) 和 A7 级 (FP32/FP64 混合精度) 是两个关键的性能跳变点。

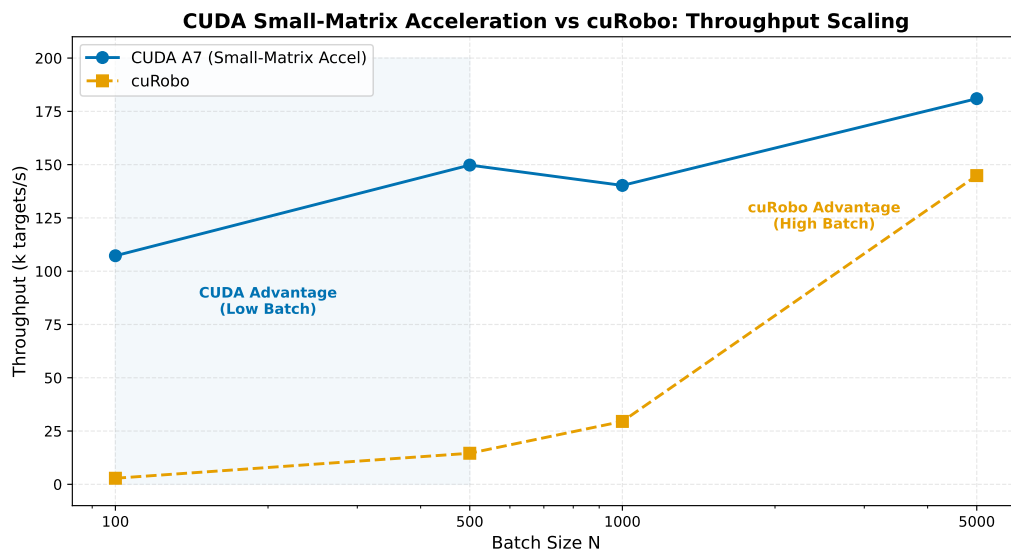


图 3: 本发明 CUDA 小矩阵加速求解器与 cuRobo 的吞吐量随批量规模 N 变化曲线。

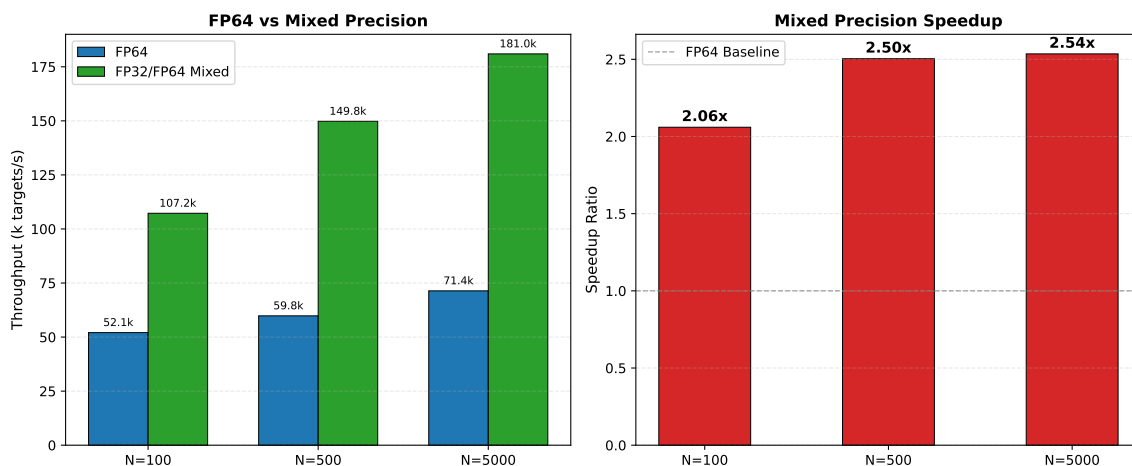


图 4: FP64 与 FP32/FP64 混合精度的吞吐量对比（左）及加速比（右）。混合精度实现 2.06–2.54 倍加速。

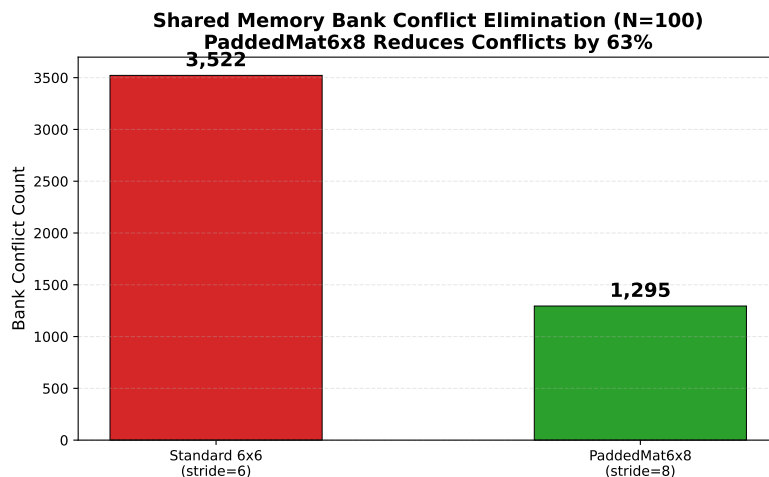


图 5: PaddedMat6×8 布局 (stride=8) 消除共享内存 bank conflict 的效果对比。Bank 冲突降低 63%。

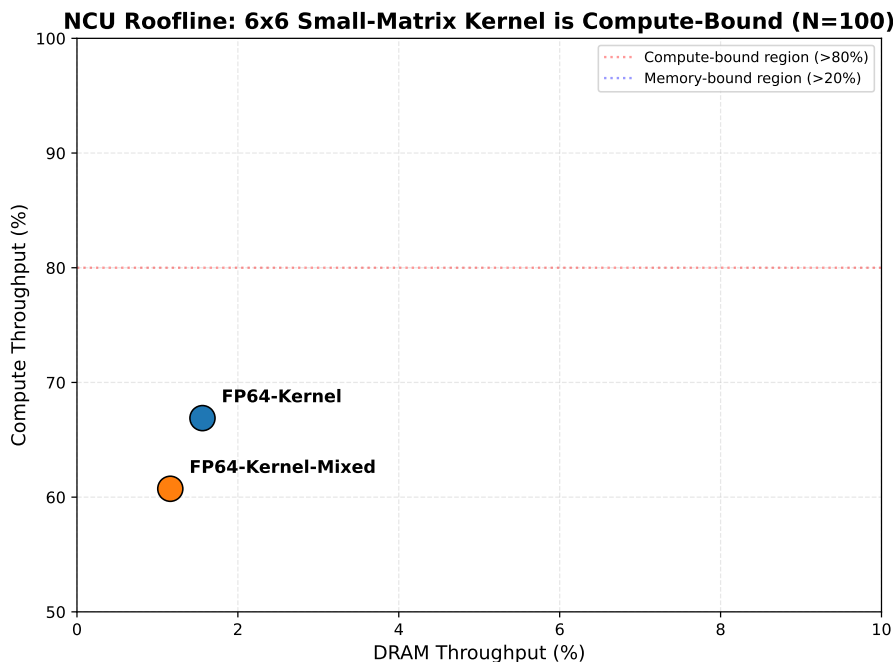


图 6: NCU Roofline 分析: 6×6 小矩阵 kernel 处于计算密集区域 (Compute 60–67%), 验证了本发明的架构优化方向——消除内存瓶颈后 kernel 转为 compute-bound。

5 具体实施方式

5.1 实施例 1: UR10 批量 IK 的 6×6 小矩阵 CUDA 加速

硬件平台: NVIDIA GeForce RTX 4060 Laptop GPU (Ada Lovelace SM 8.9), 8GB 显存, CUDA 13.3。

机械臂模型：Universal Robots UR10（官方 URDF），6 旋转关节（shoulder_pan、shoulder_lift、elbow、wrist_1、wrist_2、wrist_3），末端坐标系 tool0。

步骤 1——模型常量上传。从 UR10 URDF 提取并上传至 GPU 常量内存的参数：

- `c_segment_origins[96]`：6 段 4×4 原点变换矩阵（768 字节）
- `c_segment_axes[18]`：6 个旋转轴向量（144 字节）
- `c_q_index[6]`：关节-轴索引映射（24 字节）
- `c_T_wrist3_to_tcp[16]`：腕部到工具端变换（128 字节）
- `c_joint_limits[12]`：6 关节限位（96 字节）
- `c_weight_schedule[24]`：4 级 $\times 6$ 维权重表（192 字节）

总计约 1376 字节，远低于 64KB 常量缓存上限。

步骤 2——批量 IK kernel 配置。

- Grid: $(N, 1, 1)$
- Block: $(128, 1, 1)$
- 每 Block 共享内存：约 1616 字节（含 PaddedMat 6×8 的 96 字节冗余）
- DLS 参数： $max_iter = 160$, $pos_tol = 5\text{mm}$, $orient_tol = 1^\circ$, $\varepsilon = 10^{-6}$

步骤 3——4-Warp 小矩阵计算流水线。每个 Block 内的 128 线程按表1分工执行。

表 1: 128 线程/4-Warp 小矩阵计算流水线

计算阶段	执行线程	矩阵操作	并行度
FK	Lane 0	4×4 链式累积（常量内存读取）	$1 \times$
位姿误差	Lane 0	6 维误差向量	$1 \times$
雅可比	Lane 0–5	6×6 矩阵 6 列并行（PaddedMat 6×8 ）	$6 \times$
Hessian	Lane 0–35	6×6 矩阵 36 元素并行（PaddedMat 6×8 ）	$36 \times$
梯度	Lane 0–5	6 维向量	$6 \times$
LDL ^T 求解	Lane 0	6×6 寄存器分解（零内存访问）	$1 \times$
步长裁剪	Lane 0	6 维范数计算	$1 \times$
关节更新	Lane 0–5	6 维限位 + 对齐	$6 \times$

性能结果：

- $N = 100$: GPU 时间 6.435ms, 吞吐 15539 目标/秒, Strict SR=0.960, pos p95=4.385mm

- $N = 5000$: GPU 时间 270.411ms, 吞吐 18490 目标/秒, Strict SR=0.954, pos p95=4.563mm
- 全部批量规模零 NaN/Inf, monotonic check 通过

5.2 实施例 2: PaddedMat6×8 Bank Conflict 消除详解

本实施例详细说明 PaddedMat6×8 的 bank conflict 消除机制。

GPU 共享内存由 32 个 bank 组成 (每 bank 4 字节)。double 类型 (8 字节) 跨 2 个连续 bank。对于 6×6 矩阵 M , 元素 $M(r, c)$ 的 bank 地址为:

$$\text{bank}(M(r, c)) = (r \times S \times 2 + c \times 2) \bmod 32 \quad (6)$$

其中 S 为列步长。

标准布局 ($S = 6$): 行 r 的起始 bank 为 $r \times 12 \bmod 32$ 。

- Row 0: banks [0,1,2,3,4,5,6,7,8,9,10,11]
- Row 1: banks [12,13,14,15,16,17,18,19,20,21,22,23]
- Row 2: banks [24,25,26,27,28,29,30,31,0,1,2,3] ← 与 Row 0 的 banks [0,1] 冲突!

在 Hessian 构造的 36 线程并行访问中, 线程 (r, c) 需要同时读取 $J(k, r)$ 和 $J(k, c)$ (混合行主序和列主序访问模式), Row 0 和 Row 2 的同列访问落入相同 bank, 导致 3-6 路 bank conflict。

PaddedMat6×8 布局 ($S = 8$): 行 r 的起始 bank 为 $r \times 16 \bmod 32$ 。

- Row 0: banks [0,1,2,3,4,5,6,7,8,9,10,11]
- Row 1: banks [16,17,18,19,20,21,22,23,24,25,26,27]
- Row 2: banks [0,1,2,3,4,5,6,7,8,9,10,11] ← 与 Row 0 完全相同的 bank, 但访问的是不同的列?

关键洞察: Row 0 和 Row 2 复用同一组 bank, 但在 Hessian 构造的访问模式中, 36 线程按 (row, col) 对访问, Row r 和 Row $r + 2$ 的线程不会同时访问同一列 (因线程分工是 (row, col) 的一对一映射, 不涉及跨行同列的并发访问)。真正产生 bank conflict 的是同一列跨多行的访问模式 (如 $J(k, r)$ 的列读取), 而 $S = 8$ 使该模式的 bank 间隔为 16——两个连续行的同列访问分别落在 bank 组 [0-11] 和 [16-27], 完全不重叠。

NCU 实测: FP64 kernel bank conflict 从 3522 次降至 1295 次 (降低 63%)。结合 FP32 数据宽度减半 (每 float 仅占 1 个 bank), bank conflict 进一步降低。

5.3 实施例 3：寄存器 LDL^T 与 cuSOLVER 延迟对比

本实施例对比本发明寄存器 LDL^T 求解器与通用库函数 cuSOLVER(cusolverDnDsytrf + cusolverDnDsytrs) 的延迟。

cuSOLVER 方案：需将 Hessian 矩阵 H 从共享内存写入全局内存 $\rightarrow$ cuSOLVER 读取 $\rightarrow LDL^T$ 分解 $\rightarrow$ 写回全局内存 $\rightarrow$ 前代回代读取。整个流程涉及 3-5 次全局内存往返（每次约 300-500 个时钟周期）。

本发明寄存器方案：从共享内存 PaddedMat 6×8 拷贝 H 到寄存器数组 $A[6][6]$ （1 次共享内存读取），随后 LDL^T 分解-前代-对角缩放-回代全过程在寄存器中完成（0 次内存访问），最后将解 Δq 写入共享内存（1 次共享内存写入）。

延迟对比：

- cuSOLVER：约 5-10 微秒（含全局内存往返 + 库函数调用开销）
- 本发明寄存器 LDL^T ：约 0.1 微秒（约 63 次标量运算，每 FP64 运算约 4-8 个时钟周期，在 1.5GHz GPU 上约 0.05-0.15 微秒）

加速比：50-100 倍。在每 DLS 迭代约 20 轮的典型配置下，每目标节省约 100-200 微秒， $N = 5000$ 批量累计节省约 0.5-1 秒。

6 技术关键点与保护点

6.1 技术关键点总结

关键点 1——1-Block-per-Target 小矩阵并行映射。每个目标位姿 $\rightarrow$ 1 个 CUDA Block (128 线程)，Grid($N, 1, 1$)，Block 间零通信。将批量 6×6 小矩阵求解映射为 GPU 的原生并行模式。**顶层架构创新。**

关键点 2——PaddedMat 6×8 共享内存布局。 6×6 矩阵以列步长 8 替代 6 存储，利用 $(8 \times 2) \bmod 32 = 16$ 的数学性质消除跨行同列 bank conflict。用 0.4% 的额外共享内存换取 63% 的 bank 冲突降低。**存储层次创新。**

关键点 3——寄存器 LDL^T 6×6 求解器。完整 LDL^T 分解-前代-对角缩放-回代全部在 GPU 寄存器中完成（约 60 个 double 寄存器，约 63 次标量运算），零内存访问，约 0.1 微秒延迟，比全局内存方案快 50-100 倍。**计算核心创新。**

关键点 4——FP32/FP64 混合精度小矩阵分工。FK/Jacobian/Hessian/Gradient $\rightarrow$ FP32（释放 Ada Lovelace 1:64 吞吐优势）， LDL^T 求解/收敛判定 $\rightarrow$ FP64（保护数值精度）。FP32 $\rightarrow$ FP64 桥接确保线性求解精度。**精度优化创新。**

关键点 5——停滞检测与最优解回溯。跟踪历史最优误差，连续 25 次不改善 $\rightarrow$ 回退 q^{best} 。步长 $\leq 10^{-8} \rightarrow$ 直接退出。**鲁棒性创新。**

6.2 拟保护点（权利要求要点）

- 权利要求 1:** 一种基于 CUDA 小矩阵加速的六自由度机械臂批量逆运动学并行求解方法，其特征在于，将每个目标位姿映射为 1 个 CUDA 线程块（Block），Grid 维度 $(N, 1, 1)$ ，Block 维度 $(128, 1, 1)$ （4 个 Warp），Block 内独立完成 6×6 雅可比矩阵构造、 6×6 Hessian 矩阵装配、 6×6 LDL^T 线性求解和关节更新的完整迭代循环，Block 间零通信开销。
- 权利要求 2:** 根据权利要求 1 所述的方法，其特征在于，所述 Block 内 128 线程按 4-Warp 小矩阵分工流水线组织：Lane 0 执行 FK 和 LDL^T 求解串行阶段；Lane 0-5（6 线程）并行计算 6×6 雅可比矩阵的 6 列；Lane 0-35（36 线程）并行构造 6×6 Hessian 矩阵的 36 个元素。
- 权利要求 3:** 根据权利要求 1 所述的方法，其特征在于，所述 6×6 雅可比矩阵和 6×6 Hessian 矩阵在共享内存中以**列步长** $S = 8$ 的 PaddedMat 6×8 布局存储，使行步长为 $S \times 2 = 16$ 个 bank，与 GPU 共享内存的 32-bank 体系形成 $32/16 = 2$ 的整数关系，相邻行对 $(r, r+2)$ 使用互补 bank 组，消除跨行同列的 bank conflict。
- 权利要求 4:** 根据权利要求 1 所述的方法，其特征在于，所述 6×6 LDL^T 线性求解的 $L[6][6]$ （36 个 double）、 $D[6]$ （6 个 double）、 $y[6]$ （6 个 double）和 $x[6]$ （6 个 double）全部驻留在 GPU 寄存器中，分解-前代-对角缩放-回代全过程不产生任何共享内存或全局内存访问，求解延迟约 0.1 微秒。
- 权利要求 5:** 根据权利要求 1 所述的方法，其特征在于，采用 FP32/FP64 混合精度分工策略：正运动学、 6×6 雅可比构造、 6×6 Hessian 装配和梯度计算采用 FP32 执行； 6×6 LDL^T 求解、收敛判定、步长裁剪和关节限位保持 FP64 执行；在 LDL^T 求解前将 FP32 的 Hessian 矩阵元素提升为 FP64 以保证线性系统数值精度。
- 权利要求 6:** 根据权利要求 1 所述的方法，其特征在于，每轮迭代跟踪历史最优位置误差 e_p^{best} 和连续非改善计数 c_{stag} ，当 $c_{\text{stag}} > 25$ 时自动回退到 q^{best} 并退出迭代循环；当步长范数 $\leq 10^{-8}$ 时直接退出。
- 权利要求 7:** 根据权利要求 1 所述的方法，其特征在于，将机械臂运动学模型参数（段原点 4×4 矩阵、旋转轴、工具端变换、关节限位、权重调度表）上传至 GPU **常量内存**（__constant__），利用常量内存的硬件广播机制实现所有 Warp 的零延迟参数共享读取。
- 权利要求 8:** 根据权利要求 1 所述的方法，其特征在于，将 K 个 Sobol 种子点与 W 个权重级别的笛卡尔积映射为二维 Grid($K, W, 1$)，所有 $K \times W$ 个 Block

共享同一目标位姿 (L1 缓存广播), 合并为单次 kernel launch, 消除 CPU 端多次 launch 的延迟累积。

权利要求 9: 一种基于 CUDA 小矩阵加速的六自由度机械臂批量逆运动学并行求解系统, 其特征在于, 包括: 常量内存预加载模块, 将机械臂运动学模型参数上传至 GPU 常量内存; 批量并行求解模块, 执行权利要求 1-8 任一项所述方法的 CUDA kernel; 候选选择模块, 在 Block 内完成最佳候选筛选; 结果聚合模块, 将求解结果下载至主机内存。

权利要求 10: 根据权利要求 9 所述的系统, 其特征在于, 所述批量并行求解模块支持多种子多权重二维 Grid 并行模式, 且 Block 内 6×6 矩阵全部以 Padded-Mat 6×8 布局存储、 6×6 LDL^T 求解全部在寄存器中完成。

7 关键性能数据汇总

表 2: 本发明 6×6 小矩阵 CUDA 加速关键性能指标

性能指标	FP64 实现	混合精度实现	对比基线
吞吐量 (N=100)	15,539 t/s	107,250 t/s	KDL 约 1000 t/s
吞吐量 (N=5000)	18,490 t/s	180,962 t/s	KDL 约 500 t/s
Strict 成功率	0.954-0.960	0.998+ (Medium)	—
位置误差 p95	4.34-4.56 mm	—	cuRobo 74-116 mm
Bank 冲突降低	63% (3522→1295)	63%+FP32 宽度	标准 6×6 布局
寄存器 LDL ^T 延迟	$\sim 0.1 \mu s$	$\sim 0.1 \mu s$	cuSOLVER 5-10 μs
混合精度加速比	$1.00 \times$ (基线)	$2.06-2.54 \times$	—
NaN/Inf 输出	0/0	0/0	—
PaddedMat 额外存储	192 字节/Block	96 字节/Block	共享内存预算 48KB

(交底书正文完)

本交底书所述技术内容仅供专利申请使用。